

Systèmes d'exploitation I

Module: M3 - INF-S3 & IAP-S3

Prof. R. HANNANE

r.hannane@uca.ma

Faculté des Sciences Senglalia, Marrakech
Université Cadi Ayyad



Unix/Linux

Bref historique de Unix/Linux (1/3)

- Le système **Unics** (Uniplexed Information and Computing Service) a été créé aux laboratoires *AT&T* de *Bell*, **en 1969** sous l'impulsion de deux hommes **Ken Thompson** et **Dennis Ritchie**
- Il était **destiné à fournir aux programmeurs maison un environnement de développement auquel ils avaient goûté avec Multics** (→ **Unics** a été une version réduite de **Multics** (*Multiplexed Information and Computing Service*))
- Peu après, le nom **Unics** a été remplacé par **Unix**, et à partir de ce moment le système Unix a commencé un long chemin de développement technique
- Suite au développement par **Dennis Ritchie** du langage C, une **nouvelle version d'Unix est réécrite** (en grande partie en C) **en 1973**
⇒ **Le système Unix a connu un véritable succès**

Bref historique de Unix/Linux (2/3)

- Pour la première fois, un système d'exploitation est écrit en langage non assembleur, et devient donc portable
 - L'Université de *Californie* à *Berkeley* a obtenu une copie de la version 6 du système Unix $\Rightarrow$ AT&T et *Berkeley* **ont séparément apporté de nombreuses modifications et améliorations au système Unix**
 - Le projet **Posix** (Portable Operating System UnIX) **de normalisation du système Unix a permis de développer un standard qui définit un ensemble de procédures**
- $\Rightarrow$ Tout système Unix conforme à Posix fournit ces procédures ou appels système standards (Ces procédures constituent la *bibliothèque standard d'Unix*)

Bref historique de Unix/Linux (3/3)

- Tout logiciel écrit en n'utilisant uniquement les procédures de la norme Posix devrait fonctionner sur tous les systèmes Unix conformes à cette norme
- Une version gratuite d'Unix porte le nom de Linux (code source disponible)
- La version Linux a été créée par Linus Torvalds en 1991, et par la suite, un grand nombre de programmeurs ont contribué à son développement accéléré

Les facteurs de succès de Linux

- Le succès de Linux tient à plusieurs facteurs :
 - 1 Le code source du système, ainsi que le noyau, les programmes utilisateurs, les outils de développement **sont librement distribuables** (*licence GPL, ou GNU*)
 - 2 Linux est **compatible** avec un certain nombre de standards **Unix** au niveau du code source, incluant les spécifications POSIX, system V et BSD
 - 3 Un très grand nombre d'applications Unix gratuites disponibles sur Internet se compilent sous Linux sans aucune modification
 - 4 Le système Linux a été **développé pour les processeurs Intel** et utilise toutes les fonctionnalités de ce processeur

Caractéristiques de Linux (1/3)

Linux est un système d'exploitation :

- 1 **Gratuitement téléchargeable**
- 2 **Multiutilisateurs, multitâches:** plusieurs utilisateurs peuvent accéder simultanément au système et exécuter un ou plusieurs programmes
- 3 **Populaire:** fonctionnant sur des machines très économiques (exemple Intel 386) et reconnaisse de plus en plus de périphériques de tout genre
- 4 **Multiplateformes:** s'installe sur une machine disposant déjà d'un autre système d'exploitation de type *Windows* et il peut même reconnaître ses fichiers et les accéder

Caractéristiques de Linux (2/3)

- 5 **Interopérable:** prend en charge de nombreux système de fichiers outre que le natif *ext2fs* (**VFAT** pour la FAT 32 de Windows 95/98; **NTFS** pour Windows NT; **SO9660** pour les CDROM; **Sys V** pour Unix Système V; **MS-DOS** pour la FAT 16 de DOS et de Windows 95,...)
- 6 **Doté d'une impressionnante logithèque:**
 - ▶ **bureautique** (*Star Office* ou *Open Office*),
 - ▶ **serveur web** (*Apache*),
 - ▶ **serveur de données** (*MySQL*, *Postgree...*),
 - ▶ **serveur d'application web** (*Zope*),
 - ▶ **serveur d'impression** (*Samba*),
 - ▶ **Firewall** (filtrage de paquet par *IPTables*)
- 7 **Largement documenté:** (exemple projet LDP (Linux Documentation Project))

Caractéristiques de Linux (3/3)

- 8 **Communiquant:** prend en charge les protocoles *TCP/IP*, *AppleTalk*, *X-Window*, *NFS*, *PPP*, *ATM*...
- 9 **Système modulaire:** facile à maintenir et à faire évoluer car il est conçu en couches
- 10 **Orienté temps partagé:** les ressources du processeur et du système sont réparties entre les utilisateurs
- 11 **Portable:** disponibilité des codes sources des logiciels qui le composent
- 12 C'est un SE dont les systèmes de fichiers sont hiérarchisés en arbre: plusieurs systèmes de fichiers peuvent être rattachés au système de fichiers principal $\Rightarrow$ ce qui rend le système indépendant du matériel et en assure la portabilité

Les distributions Linux

- **Linux** est le plus souvent diffusé sous forme d'une distribution
 - Une **distribution** est un ensemble cohérent de logiciels assemblé sous le noyau **Linux/GNU** et formant un système d'exploitation complet
 - Il existe de nombreuses distributions Linux, et chacune possède un fonctionnement qui lui est propre, et répondant à des objectifs variés
 - Chacune des distributions a ses avantages et ses inconvénients
- ⇒ **Débuter avec Linux**, c'est surtout choisir une distribution qui corresponde avec les usages qu'on attend du système

Les distributions Linux

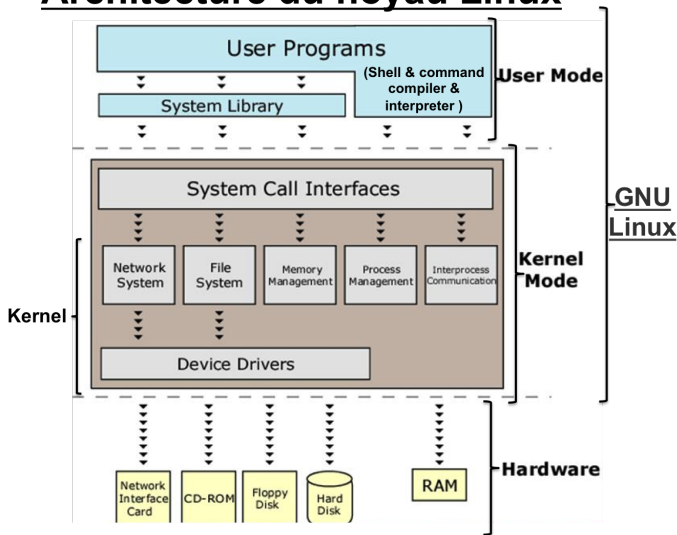
- Les **principales distributions de Linux** sur le marché:



...

Architecture logicielle de Linux

Architecture du noyau Linux



Structure de l'architecture logicielle de Linux

- Le système GNU/Linux est structuré essentiellement de :
 - ▶ **Espace Noyau** avec des interfaces et drivers de périphérique
 - ▶ **Espace utilisateur** avec Programme système (shell, compilateur, commande...)
 - L'interface entre les programmes des utilisateurs et le noyau est définie par un ensemble de procédures et de fonctions, soit directement au niveau du noyau, soit par l'intermédiaire de bibliothèques
 - Le noyau assure la gestion des ressources physiques (processeur, mémoires, périphériques,...) et logicielles (processus, fichiers...)
- ⇒ Le noyau réside entre interface appels systèmes et hardware
→ L'utilisateur n'accède pas directement au noyau mais à un interpréteur de commandes

Les appels système

- Certains **traitements ne peuvent pas être exécutés et contrôlés directement par un programme utilisateur** car le système ne le permet pas
⇒ Le **programmeur doit faire explicitement appel aux services du système d'exploitation pour accéder à certaines ressources**
- À ces fins, le **système d'exploitation propose une interface de programmation**, c'est-à-dire qu'il **permet d'accéder à un certain nombre de fonctionnalités** qu'il exécute pour l'utilisateur
- Les **appels système** sont les **interfaces proposées par le système d'exploitation pour accéder aux différentes ressources de la machine**

Les appels système

Leurs but

- Un **appel système** a pour but de :
 - ▶ **Changer de mode d'exécution** pour passer du mode utilisateur au mode maître
 - ▶ **Récupérer les paramètres et vérifier la validité de l'appel**
 - ▶ **Lancer l'exécution de la fonction demandée**
 - ▶ **Récupérer la (les) valeur(s) de retour**
 - ▶ **Retourner au programme appelant** avec retour au mode utilisateur
 - **Pour chaque composant de l'ordinateur, le système d'exploitation propose des appels système**
- ⇒ **Seuls ces appels systèmes peuvent être utilisés, le reste des primitives du système est protégé et leur usage direct est interdit aux utilisateurs**

Les appels système

Types d'appels systèmes

- Les appels système sont généralement **classés en 5 catégories** :
 - ▶ **Contrôle de processus**
 - charger, exécuter, créer, terminer des processus, obtenir, signaler, des événements, libérer de la mémoire, etc
 - ▶ **Manipulation des fichiers**
 - créer, supprimer, ouvrir, fermer, lire, écrire, repositionner, etc
 - ▶ **Gestion de périphériques**
 - demander, libérer, obtenir, attacher, etc
 - ▶ **Entretien d'information**
 - obtenir, définir l'heure ou la date, définir les données du système, etc
 - ▶ **Communications**
 - créer, supprimer des connexions de communication, envoyer, recevoir de messages, transférer des informations sur les états, etc

Les appels système

Exemple d'utilisation

- Il est nécessaire de **faire appel au système d'exploitation pour créer un fichier** sur le disque dur et cela via un appel système comme **createfile** sous Windows 2000
- Il est logique d'interdire au programme utilisateur d'effectuer lui-même la création pour les raisons suivantes:
 - ▶ Le **système d'exploitation** doit vérifier si l'utilisateur dispose du droit d'écriture dans le répertoire, autrement il refusera de créer le fichier
 - ▶ Il doit également vérifier si un fichier du même nom n'existe pas déjà
 - ▶ La création du fichier nécessite l'attribution de quelques secteurs du disque dur pour l'enregistrement des données du fichier → *L'utilisateur n'a aucune connaissance des zones libres du disque*

Les appels système

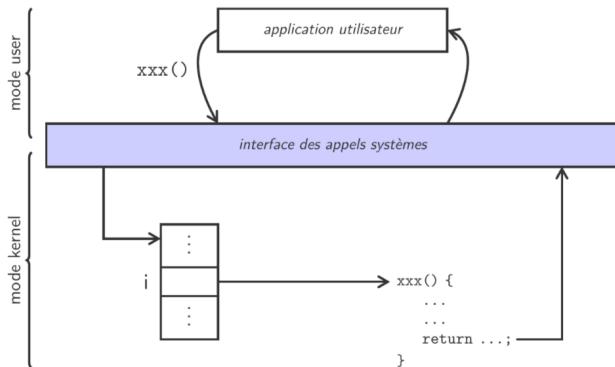
Exécution d'un appel système

- Lorsqu'un **programme** effectue un **appel système**, son exécution en mode utilisateur est interrompue et le **système** prend le contrôle en mode superviseur
- ⇒ **Un appel système est exécuté en mode noyau même si le programme ayant demandé son exécution est exécuté en mode utilisateur**
- Gestion via une **interface des appels systèmes**:
 - ▶ Fais le relais en interceptant les appels systèmes dans l'API
 - ▶ Utilise une **table associative** numérotant les appels systèmes
- Le **programme utilisateur** ignore les détails de l'appel système
 - ▶ Il doit respecter l'API et passer les bons paramètres
 - ▶ Il doit comprendre la valeur de retour

Les appels système

Exécution d'un appel système

- Sur la majorité des systèmes d'exploitation, **les appels système peuvent être utilisés comme de simples fonctions écrites en C, leur exécution donne en retour un résultat**. Si l'appel échoue, le résultat vaut -1



Exécution d'un appel système

Exemple d'algorithme d'un appel système pour la copie d'un fichier

❶ Récupérer le nom du fichier source

- ▶ Afficher une phrase sur la sortie standard
- ▶ Lire l'entrée standard

❷ Récupérer le nom du fichier destination

- ▶ Afficher une phrase sur la sortie standard
- ▶ Lire l'entrée standard

❸ Ouvrir le fichier source, créer et ouvrir le fichier destination

- ▶ **Tant que la lecture n'échoue pas**
 - Lire un caractère depuis la source
 - Écrire le caractère dans la destination

❹ Fermer les fichiers source et destination

Exécution d'un appel système

Exemple de code d'un appel système pour la copie d'un fichier

Version simplifiée de la copie d'un fichier

```
void copy ( char * src , char * dst )
{
    char buffer [ BUFFSIZE ];
    int srcFile = open ( src , O_RDONLY ) ;
    int dstFile = open ( dst , O_WRONLY | O_CREAT , 0666 ) ;
    if ( srcFile != -1 && dstFile != -1)
    {
        int r ;
        while ( ( r = read ( srcFile , buffer , BUFFSIZE ) ) )
        {
            write ( dstFile , buffer , r ) ;
        }
        close ( srcFile ) ;
        close ( dstFile ) ;
    }
}
```

Systèmes de fichiers de Linux

Quelques informations utiles

Notes à propos des **fichiers** :

- Un **fichier** est un **ensemble de données enregistrées de façon à être lues et traitées par ordinateur**
- Les fichiers **peuvent être regroupés dans des répertoires**
- Un **répertoire** peut contenir soit des fichiers, soit d'autres répertoires dans une **structure arborescente**
- Les **appels système** permettent de créer les fichiers et les répertoires, ainsi que de les supprimer, de les ouvrir, de les lire et de les modifier

Quelques informations utiles

Types de fichiers

- Le système **Linux ne connaît que 3 types de fichiers**:
 - ❶ **Les fichiers ordinaires(-)**: contiennent des données,(ex: fichiers de texte, binaires, images...). Ils servent à mémoriser les programmes et les données des utilisateurs et du système
 - ❷ **Les fichiers répertoires(d)**: sont les **nœuds de la structure arborescente** des fichiers. Ils correspondent à des sous-bibliothèques ou dossiers qui contiennent d'autres fichiers
 - ❸ **Les fichiers spéciaux**: ils sont **liés à un périphérique et correspondent à :**
 - ▶ **des programmes(b,c)** (pilotes)qui gèrent les échanges avec les périphériques (disques, terminaux, imprimantes, lecteurs de bandes, etc)
 - ▶ **des tubes et sockets(p,s)**, utilisés pour les liaisons interprocessus (la communication entre processus)
 - ▶ **des liens symboliques(l)** (fichiers "pointant" sur un autre fichier)

Quelques informations utiles

Nomenclature des fichiers

- Pour donner un nom à un fichier, il faut suivre quelques règles simples
- Ces règles sont valables pour tous les types de fichiers:
 - ▶ Sur les anciens systèmes Unix **un nom de fichier ne pouvait pas dépasser 14 caractères**. Sur les systèmes actuels, dont Linux, **on peut aller de 1 jusqu'à 255 caractères**
 - ▶ Linux fait la **distinction entre les noms de fichiers en minuscules et en majuscules** (ex: Toto, TOTO, ToTo et toto sont des noms de fichiers différents, avec un contenu différent)
 - ▶ **La plupart des caractères** (les chiffres, les lettres, les majuscules, les minuscules, certains signes, les caractères accentués) **sont acceptés, y compris l'espace**
 - ▶ **Quelques caractères sont à éviter** car ils ont une signification particulière au sein du shell : `& ; ( ) ~ / | ' ?`

Systèmes de fichiers de Linux

- Un **système de fichiers** est une structure logique qui permet au système d'exploitation d'accéder aux fichiers qui se trouvent dans un périphérique de données (Disque dur, clé USB, etc).
- Il est **créé au moment de l'opération de formatage**
⇒ il correspond au **type de formatage** qui va être appliqué à une partition,
(*Il définit la manière dont les fichiers sont nommés, stockés et récupérés sur un périphérique de stockage*)
- Les systèmes d'exploitation utilisent des systèmes de fichiers différents tel que *FAT, ExFAT, NTFS, HFS, HFS+, APFS, Ext4,...*
⇒ **L'organisation et le fonctionnement de ces systèmes de fichiers est différent** ou certains peuvent être plus performants dans des dossiers avec des milliers de fichiers, d'autres dans le recouvrement dans le cas de corruption...
- **Ext** est le système de fichiers par défaut sur les systèmes Linux
⇒ **Ext4** gère parfaitement de nombreux petits fichiers

Systèmes de fichiers de Linux

Système de fichiers hiérarchisés

- **Le système de fichiers de Linux est à géométrie variables**, il est compatible avec la plupart des systèmes de fichiers existant (et entre autre les systèmes FAT, FAT 32 ou NTFS de Windows de Microsoft), **il peut gérer la plupart des support physiques, et il peut s'agrandir à tout moment** (l'administrateur du système peut créer un nouveau point de montage n'importe où dans l'arborescence)
- **L'organisation du système de fichiers est une organisation logique basée sur une organisation physique constituée d'un ou plusieurs disques**
 - ▶ Chacun est divisé en une ou plusieurs partitions physiques et l'espace disque physique disponible est ainsi découpé en **disques logiques**
 - ▶ Ces disques sont découpés en série de groupe de blocs, ou chaque groupe de blocs est composé de la même manière
 - ▶ Les groupe de blocs du disque stockent les **inodes** (les métadonnées des fichiers et dossiers comme le type de fichiers, permissions, etc) ainsi que les **données du fichier**

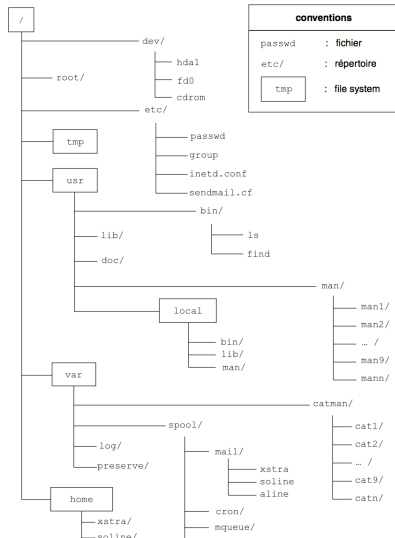
Systèmes de fichiers de Linux

Système de fichiers hiérarchisés

- Initialiser un disque logique consiste à créer sur ce disque un système de fichiers appelé **file system**
- **L'arborescence de fichiers** est construite au démarrage du système à partir de plusieurs **file system**
- En général, **plusieurs systèmes de fichiers** sont "**montés**" sur le **système "racine"**
⇒ Celui qui contient le répertoire " / "
- Un file system particulier, le **root file system** contient la racine de l'arborescence, le noyau, les fichiers systèmes, et des répertoires vides (*/tmp/, /usr/, ... sur lesquels seront greffées des arborescences de fichiers se trouvant dans d'autres file systems*)

Systèmes de fichiers de Linux

Organisation hiérarchisée



Systèmes de fichiers de Linux

Système de fichiers hiérarchisés

Répertoire	Description
/	Répertoire racine, point d'entrée au système de fichiers
/boot	Contient le noyau Linux et l'amorceur
/bin	Contient les exécutables de base:ls,cp,mv...
/dev	Contient des fichiers spéciaux représentant les périphérique de la machine
/etc	Contient les fichiers de configuration du système
/home	Contient les fichiers des utilisateurs(répertoire par utilisateur)
/lib	Contient les bibliothèques et les modules du noyau (/lib/modules)
/media	Point de montage des média (CD,USB...)
/root	Répertoire personnel de l'administrateur
/sbin	Contient les exécutables relatifs à l'administration du système
/tmp	Contient les fichiers temporaires
/usr	Contient les exécutables des programmes(/usr/bin et /usr/sbin) la documentation (usr/doc)
/var	Contient les fichiers relatifs à la maintenance du système

Systèmes de fichiers de Linux

Structure d'un file system

- La structure d'un file system est globalement la même dans toutes les versions d'Unix/Linux: Chaque file system est composé d'un grand nombre de blocs contigus
- Sa structure de base est comme suite :

Bloc	Contenu
0	contient le boot et l'identification du disque
1	appelé aussi <i>superbloc</i> contient des informations sur le file system (<i>date de dernière mise à jour du file system, le nombre de fichiers qu'il peut contenir, sa taille, un pointeur sur les blocs libres</i>)
2 à k	contiennent la table des inodes
à partir du bloc k+1	sont stockées les blocs de données

Systèmes de fichiers de Linux

Les inodes

- Sous Unix/Linux, un **fichiers est une suite d'octets non structurée**
- ⇒ **Toutes les informations sur le fichier qui permettent de garder la trace de tous les fichiers et répertoires d'un système de fichiers basé sur Linux ou UNIX, sont regroupées dans une structure de données appelée **inode****
- **Un fichier n'est pas repéré par son nom mais par le numero (**index**) de l'inode qui le caractérise**
 - ⇒ Dans **ext4** de Linux, le nom de fichier est stocké dans un bloc de répertoire Ce dernier contient le numéro d'inode, le nom et la longueur du nom du fichier.
- **Cet index variant de 1 à n permet de retrouver l'inode dans la **table des inodes****
 - ⇒ Aucun fichier n'utilise le numéro 1
 - ⇒ Le numéro 2 représente la racine du file system
- **Le nombre maximum de fichiers par file system est égal au nombre d'inodes alloués à l'initialisation**

Systèmes de fichiers de Linux

Les inodes

- Les informations contenues dans un inode sont:

- ❶ **Bloc de disque** où la donnée est stockée
- ❷ **Type de fichier**: (*fichier ordinaire, répertoire,...*)
- ❸ **Bits de permissions**: `suid,sgid,t,rwx rwx rwx`
- ❹ **Nombre de liens**
- ❺ **Numéro de propriétaire (UID)**
- ❻ **Numéro de groupe (GID)**
- ❼ **Taille du fichier en octets**
- ❽ **Des pointeurs** vers les blocs de données correspondants
- ❾ **Date et heure du dernier accès**
- ❿ **Date et heure de la dernière modification**
- ⓫ **Date et heure de la dernière modification de l'inode**

Systèmes de fichiers de Linux

Les inodes: création et fonctionnement

- Le système de fichiers **lors du formatage d'une partition de disque en ext4 crée chaque inode** qui possède un numéro unique au sein du système de fichiers
- **Le nombre total d'inodes d'un système de fichiers est défini manuellement lors de sa création**
⇒ Il indique le nombre maximal de fichiers qu'un système de fichiers peut supporter
- **Lors de la création ou de la copie d'un fichier, Linux attribue un numéro d'inode différent au nouveau fichier**
- **Lors du déplacement d'un fichier, le numéro d'inode ne change que si le fichier est déplacé vers un système de fichiers différent**

Systèmes de fichiers de Linux

Les chemins

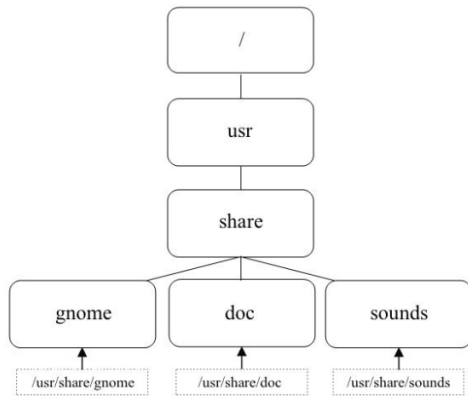
- Un **chemin** permet de définir un emplacement au sein du système de fichiers
- Tous les fichiers peuvent être **atteints par un chemin** dans la hiérarchie à partir du dossier racine $\Rightarrow$ c'est la liste des répertoires et sous-répertoires empruntés pour accéder à un fichier dans un endroit donné: **de l'arborescence jusqu'à la position souhaitée** (répertoire, fichier)
- Dans chaque répertoire, on trouve au moins deux fichiers, nommés **.** (point) et **..** (point point)
 - ▶ Le premier, qui a pour symbole **.** permet de référencer le répertoire lui-même (répertoire courant)
 - ▶ Le second, qui a pour symbole **..** permet d'accéder au **répertoire parent** (répertoire situé juste au dessus)
- A chaque instant, toute tâche possède un répertoire courant, ou répertoire de travail

Systèmes de fichiers de Linux

Les chemins

- **Exemple:** On prend la figure suivante:
On remarque qu'il y a un répertoire nommé "sounds" qui se trouve dans le répertoire "share" qui se trouve lui-même dans le répertoire "usr" qui se trouve lui-même dans "racine"

⇒ Pour accéder au répertoire "sounds" on spécifie son chemin comme suit:
`/usr/share/sounds`
où **le premier "/"** représentent **le répertoire racine**, **les autres "/"** sont utilisés pour **séparés les noms des répertoires**



Systèmes de fichiers de Linux

Les chemins

- Un **nom de fichier** est ainsi généralement **complété par son chemin d'accès**

⇒ C'est ce qui fait que le fichier *toto* du répertoire *rep1* est différent du fichier *toto* du répertoire *rep2*

- Pour désigner un fichier quelconque, on peut utiliser soit un :
 - a) **chemin absolu**
 - b) **chemin relatif**

Systèmes de fichiers de Linux

Les chemins: a) chemin absolu

- Un **chemin absolu** permet d'accéder à un fichier quelconque dans l'arborescence du système de fichiers
- Il est composé d'une suite de noms de répertoires à traverser en partant de la racine, séparés par le caractère / (et non \ comme sous DOS) et se termine par le nom du fichier que l'on veut atteindre
- La **longueur du chemin d'accès absolu** d'un fichier est **limitée à 1024 caractères**

Systèmes de fichiers de Linux

Les chemins: a) chemin absolu

A retenir:

- **Un chemin absolu ou complet :**

- ▶ **Démarre de la racine**, donc commence par un /,
- ▶ **Décrit tous les répertoires à traverser** pour accéder à l'endroit voulu
- ▶ **Ne contient pas de . ni de ..**

⇒ **Example:**

Le chemin " /usr/bin/compress" désigne le fichier *compress*, qui se trouve dans le répertoire *bin*, lui-même dans le répertoire *usr* de la racine

Le premier caractère / indique qu'il s'agit d'un chemin absolu

Systèmes de fichiers de Linux

Les chemins: b) chemin relatif

- Un nom de **chemin** peut aussi être **relatif** à sa **position courante** dans le répertoire
- Le **système** (ou le **shell**) mémorise la **position actuelle** d'un utilisateur dans le **système de fichier**, le répertoire actif
- On peut accéder à un autre répertoire de l'arborescence depuis l'emplacement actuel sans taper le chemin complet
⇒ **Uniquement en précisant le chemin le plus court relativement à votre position actuelle au sein de l'arborescence**
- Il faut souvent **utiliser deux entrées** particulières de **répertoires** :
 - ▶ **Le point .** représente le répertoire courant, actif. Il est généralement implicite
 - ▶ **Les doubles points ..** représentent le répertoire de niveau inférieur

Systèmes de fichiers de Linux

Les chemins: b) chemin relatif

A retenir:

- Un chemin relatif :
 - ▶ **Décrit un chemin relatif à une position donnée** dans l'arborescence, généralement (mais pas toujours) **depuis la position courante**
 - ▶ **Décrit en principe le plus court chemin** pour aller d'un point à un autre
 - ▶ **Peut contenir le point . ou les doubles points ..**